OpenMP — Lesson 5

Load balancing: the `schedule` clause

Gustavo Lozano

November 2026

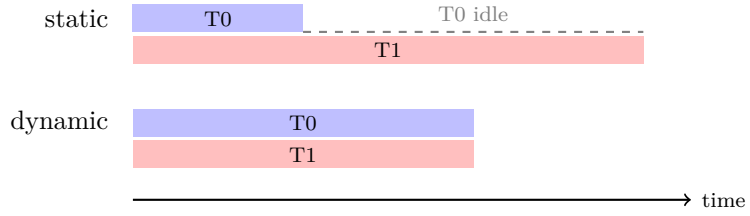
Sources: Chapman, Jost & van der Pas [1], Ch. 4 "OpenMP Language Features" (the `schedule` *and* `nowait` *clauses) and Ch. 5 "How to Get Good Performance" (load imbalance).*

1 The idea

In Lesson 4 every iteration cost the same, so cutting the loop into p equal blocks was perfect. Now suppose iteration i costs work proportional to i. With two threads and equal blocks, thread 0 gets iterations $0, \ldots, 499$ and thread 1 gets $500, \ldots, 999$. Thread 1 has about **three times** more work:

$$\sum_{i=0}^{499} i \approx \tfrac{1}{4} \sum_{i=0}^{999} i, \qquad \sum_{i=500}^{999} i \approx \tfrac{3}{4} \sum_{i=0}^{999} i.$$

Thread 0 finishes early and sits idle at the barrier, while thread 1 is still working. The loop takes as long as the *slowest* thread. This is **load imbalance**.



2 The `schedule` clause

`schedule(kind[, chunk])` on an `omp for` decides which thread gets which iterations.

Kind	How iterations are assigned	Good for
`static`	split into p contiguous blocks *before* the loop starts (with `chunk`: blocks of size `chunk` dealt round-robin)	equal-cost iterations; lowest overhead
`dynamic, chunk`	a thread takes `chunk` iterations; when done, it asks for the next free chunk (default chunk = 1)	unequal or unpredictable cost
`guided`	like `dynamic`, but chunks start large and shrink as the loop progresses	unequal cost, fewer requests

The `nowait` clause. The `omp for` loop normally ends with an implicit barrier. `nowait` removes it, so a thread that finishes its share continues immediately. Here we use it only to *measure* when each thread finishes; the end of the `parallel` region still has its own barrier.

3 The code

The program runs the same unequal loop three times, once per schedule, and prints when each thread finished and the total time.

schedule.c

```c
/*
 * schedule.c -- OpenMP Lesson 5: how loop iterations are assigned to threads.
 * Source: Chapman, Jost & van der Pas (2007), "Using OpenMP",
 *         Ch. 4 (the schedule clause, the nowait clause),
 *         Ch. 5 (load imbalance).
 *
 * Compile (no -O flag, so work() is not optimised away):
 *     gcc -Wall -fopenmp -o schedule schedule.c
 * Run (use as many threads as you have cores, see `nproc`):
 *     OMP_NUM_THREADS=2 ./schedule
 */

#include <stdio.h>
#include <omp.h>

#define N    1000      /* number of iterations                       */
#define K    200       /* iteration i does i*K units of work         */
#define MAXT 64        /* max threads we keep results for            */

/* Unequal work: iteration 999 costs ~1000x more than iteration 1. */
static void work(int i)
{
    volatile double x = 0.0;           /* volatile: keep the loop alive */
    for (long j = 0; j < (long)i * K; j++)
        x += 1.0;
}

/* Print when each thread finished its share, and the total time. */
static void report(const char *name, const double finish[], int p, double total)
{
    printf("%-18s total %.3f s   finish per thread:", name, total);
    for (int k = 0; k < p; k++)
        printf(" %.3f", finish[k]);
    printf("\n");
}

int main(void)
{
    int p = omp_get_max_threads();
    double finish[MAXT];
    double t0;

    /* ---- 1) static: iterations cut into p contiguous blocks BEFORE the
       loop starts. Thread 0 gets the cheap iterations, the last thread
       the expensive ones -> load imbalance.                          */
    t0 = omp_get_wtime();
    #pragma omp parallel
    {
        #pragma omp for schedule(static) nowait
        for (int i = 0; i < N; i++)
            work(i);
        /* nowait removed the barrier after the loop, so each thread
           records the moment IT finished, not when the slowest did.   */
        finish[omp_get_thread_num()] = omp_get_wtime() - t0;
    }
    report("static", finish, p, omp_get_wtime() - t0);

    /* ---- 2) dynamic, chunk 1: a thread takes ONE iteration, and when it
```

```
       is done it asks for the next free one. Threads that got cheap work
       simply take more iterations -> balanced. Cost: one request per
       iteration (a little overhead).                                    */
    t0 = omp_get_wtime();
    #pragma omp parallel
    {
        #pragma omp for schedule(dynamic, 1) nowait
        for (int i = 0; i < N; i++)
            work(i);
        finish[omp_get_thread_num()] = omp_get_wtime() - t0;
    }
    report("dynamic,1", finish, p, omp_get_wtime() - t0);

    /* ---- 3) guided: like dynamic, but chunks start large and shrink.
       Fewer requests than dynamic,1 and still balanced at the end.      */
    t0 = omp_get_wtime();
    #pragma omp parallel
    {
        #pragma omp for schedule(guided) nowait
        for (int i = 0; i < N; i++)
            work(i);
        finish[omp_get_thread_num()] = omp_get_wtime() - t0;
    }
    report("guided", finish, p, omp_get_wtime() - t0);

    return 0;
}
```

```
gcc -Wall -fopenmp -o schedule schedule.c
OMP_NUM_THREADS=2 ./schedule
```

Typical output on 2 cores (your times will differ):

```
static             total 0.156 s   finish per thread: 0.063 0.155
dynamic,1          total 0.102 s   finish per thread: 0.102 0.102
guided             total 0.105 s   finish per thread: 0.105 0.105
```

Key ideas

- With `static`, thread 0 finishes at about 1/3 of the time of thread 1: exactly the 1/4 vs. 3/4 split of the work computed above.
- With `dynamic` and `guided`, all threads finish together. Nobody is idle, so the total time drops.
- The schedule does not change the *result*, only *who* does *which* iteration and therefore the time.
- `static` is decided once, at no cost. `dynamic` pays a small cost every time a thread asks for a new chunk. Use it when the cost of iterations varies.

4 The experiment

Experiment

Question: which schedule balances unequal work, and what does balancing cost?

1. Run with 1 thread and with $p =$ `nproc` threads:

```
OMP_NUM_THREADS=1 ./schedule
OMP_NUM_THREADS=$(nproc) ./schedule
```

2. For $p =$ `nproc`, compare the per-thread finish times of each schedule.
3. Compare the three totals in the 1-thread run.

What to observe and answer.

1. With `static`, which thread finishes last? Predict the ratio between the first and last finish time from the work formula, and check it.
2. Why do `dynamic` and `guided` make all threads finish at the same time?
3. With 1 thread there is nothing to balance, yet `dynamic` and `guided` are a bit slower than `static`. Why?

Answers (read after trying). (1) The last thread, because it owns the most expensive iterations. With p threads, block k holds a share of the work proportional to $2k + 1$, so with 2 threads the ratio is $1 : 3$, and with 4 threads $1 : 3 : 5 : 7$. (2) Work is handed out on demand. A thread that drew cheap iterations comes back sooner and takes more of them, so all threads stay busy until the loop runs out. (3) Scheduling overhead. `dynamic,1` makes one request per iteration (1000 requests) even when there is no other thread to share with. `static` decides everything once, before the loop. The general rule: use `static` when iterations cost the same, `dynamic` or `guided` when they do not.

Next lesson. Beyond loops: the `sections` and `single` constructs, and the explicit `barrier` (Chapman et al., Ch. 4).

References

[1] B. Chapman, G. Jost, and R. van der Pas. *Using OpenMP: Portable Shared Memory Parallel Programming*. MIT Press, Cambridge, MA, 2007.